

Procesamiento de Datos Masivos

Clase 06 - Map-Reduce y Apache Spark

Qué haremos hoy

- Conoceremos las motivaciones para el desarrollo y uso de Map-Reduce
- Entenderemos su funcionamiento
- Aplicaremos Map-Reduce para realizar consultas SQL de forma distribuida

Contexto



La web y los buscadores

- Sabemos cómo hacer búsquedas eficientes en datos estructurados aprovechando índices
- Qué pasa en páginas web (búsqueda documental / datos no estructurados)?
- Internet creció más rápido que nuestra capacidad de organizarlo

La web y los buscadores

- Primeros motores: matching palabras claves y frecuencia de términos (term frequency)
- Problemas:
 - Manipulación de la información
 - Resultados poco relevantes
 - Estrategias no escalaban a la velocidad en que lo hacía el Internet

La web y los buscadores

Consulta: “**mejor universidad de Chile**”

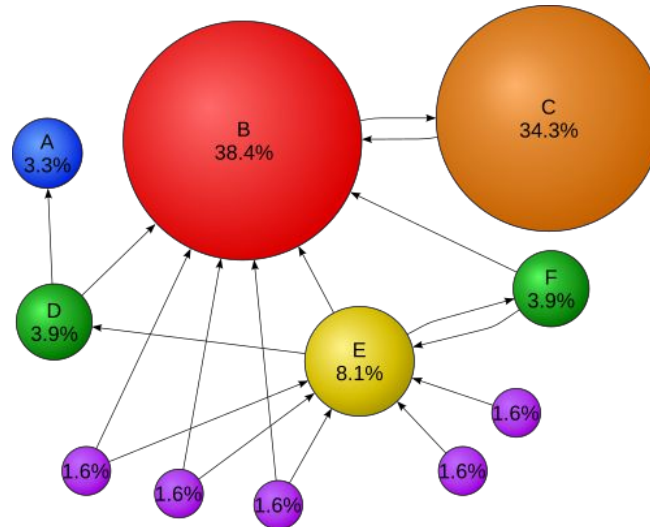
- Página 1 - **spam**: Una página que repite muchas veces las palabras “universidad” y “Chile”
- Página 2 - **legítima**: Información real y es citada por otras páginas
- Página 3 - **blog**: “Las mejores universidades de Chile según mi opinión”

La web y los buscadores

- No basta con encontrar coincidencias, hay que ordenar resultados
- ¿Y cómo ordenamos?

Google y PageRank

- Larry Page y Sergey Brin: “los links son votos”
- No solo importa tu contenido, importa quién te cita



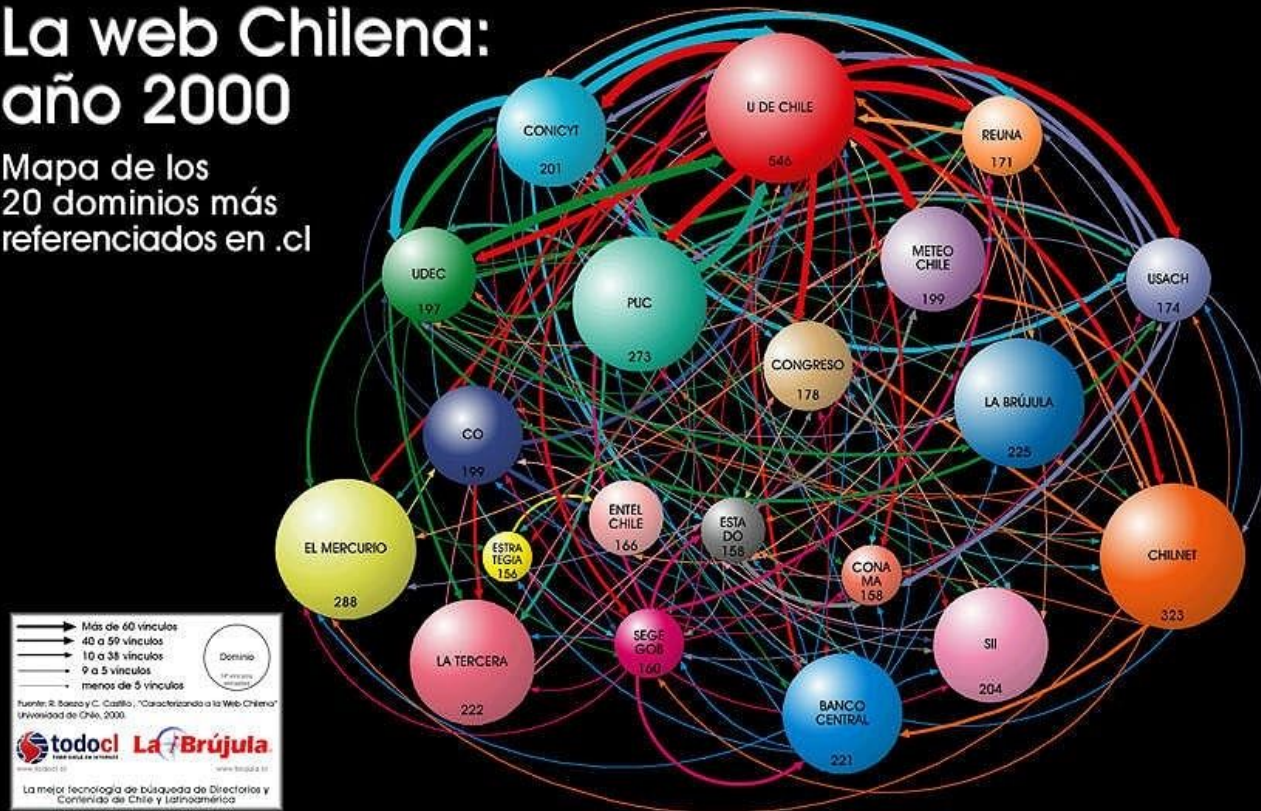
Google y PageRank

Consulta: “**mejor universidad de Chile**”

- Página 1 - **spam**: Una página que repite muchas veces las palabras “universidad” y “Chile”
- Página 2 - **legítima**: Información real y es citada por otras páginas
- Página 3 - **blog**: “Las mejores universidades de Chile según mi opinión”

La web Chilena: año 2000

Mapa de los
20 dominios más
referenciados en .cl



Google y PageRank

- El Internet deja de ser un conjunto de textos aislados
- Estrategia nos permite reducir el ruido del spam
- Surge el **grafo del Internet** y búsquedas permiten encontrar **información relevante!!**
- Pero...



Desafío de PageRank

- Tenemos un grafo **ENORME** que crece día a día
- Cálculo iterativo
- ¿Cómo procesamos esto?

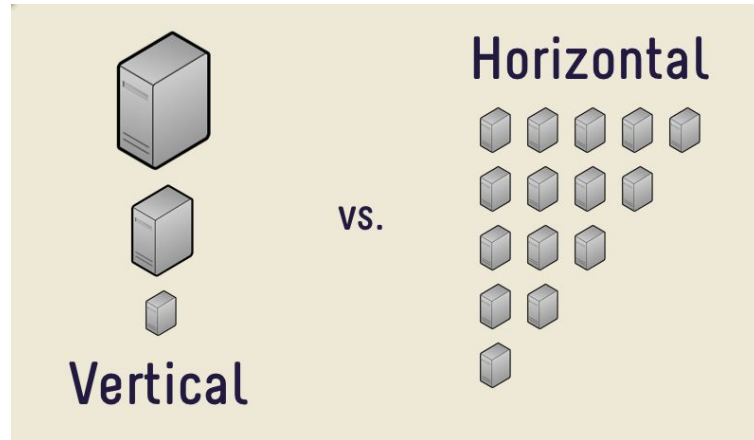


Map-Reduce



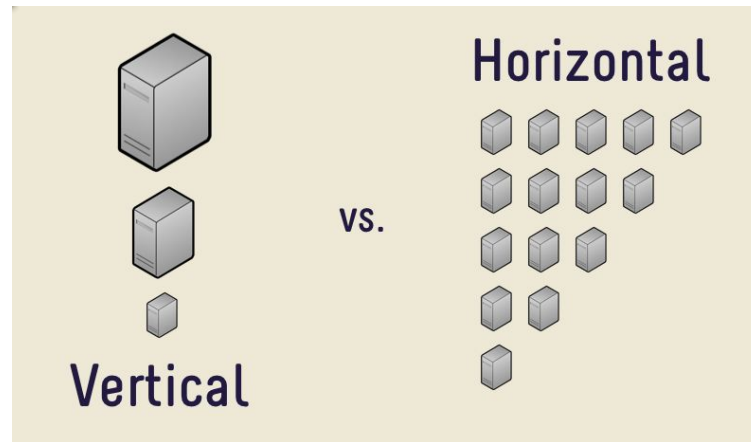
Procesamiento a gran escala

- Nuestro problema no es almacenamiento, es poder de cómputo
- ¿Y si escalamos horizontalmente?
- Algunos beneficios:
 - Menor costo
 - Evitamos fallas catastróficas



Procesamiento a gran escala

- ¿Y por dónde perdemos?
 - Costo de comunicación
 - Complejidad de tener información repartida
 - Falla de nodos de cómputo



Map-Reduce

Modelo de programación para procesar:

- Datos masivos
- En paralelo
- Aprovechando múltiples máquinas

Además de ser una **forma de pensar los problemas**

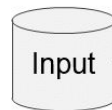
Map-Reduce

1. **Map**
2. Shuffle
3. **Reduce**

E iterar si es necesario

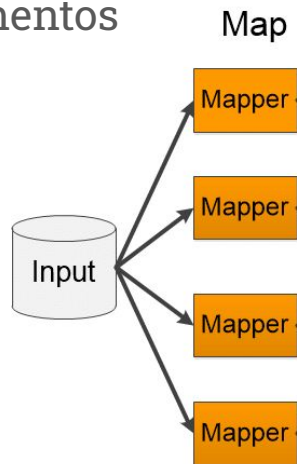
Ejemplo

- Tenemos millones de documentos
- Queremos contar cuántas veces aparece cada palabra en todos los documentos
- Tomemos:
 - Doc1: "data engineering data"
 - Doc2: "big data systems"
- Y llevemos cada doc a un nodo de cómputo



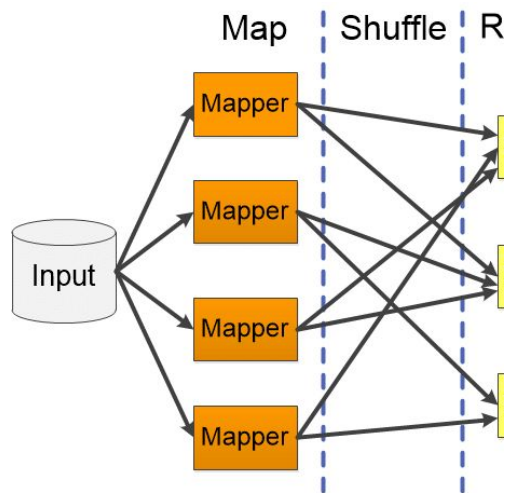
Ejemplo - Map

- La función map transforma cada elemento en **pares llave-valor**
- APlicamos la misma función a todos los elementos
 - Doc1: "data engineering data" →
(data,1), (engineering,1), (data,1)
 - Doc2: "big data systems" →
(big, 1), (data, 1), (systems, 1)



Ejemplo - Shuffle

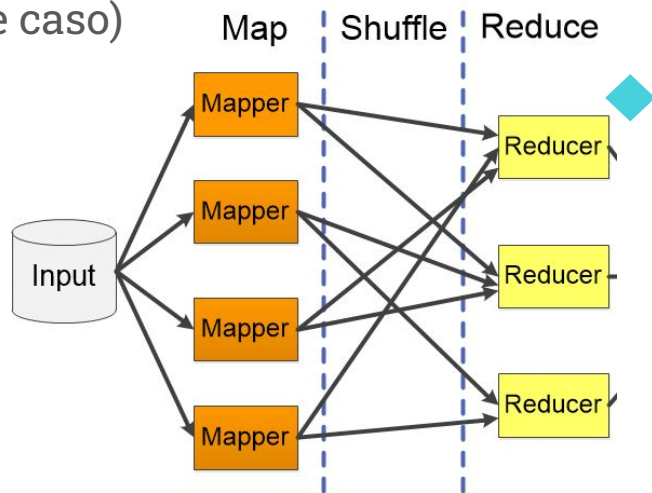
- Reordenamos los datos entre los nodos
- Mismas llaves deben quedar en un mismo nodo
- Requiere una coordinación y tráfico de red
 - [(data,1), (data,1), (data, 1)]
 - [(engineering,1)]
 - [(big, 1)]
 - [(systems, 1)]



Ejemplo - Reduce

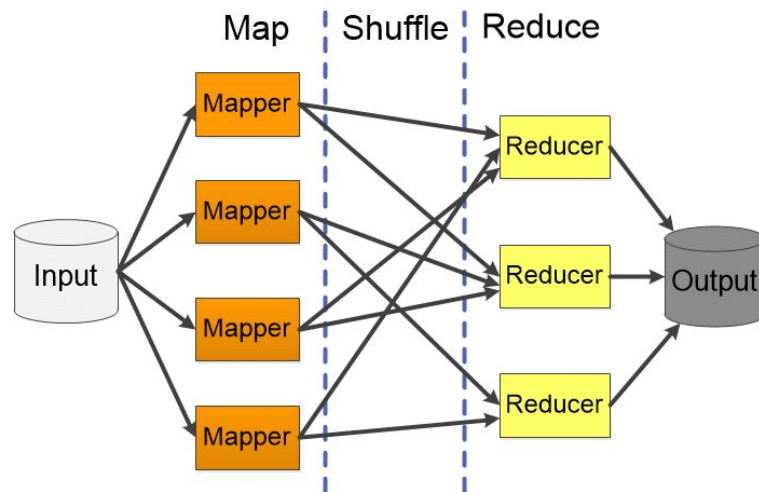
- Aplicamos la agregación
- En este caso, agrupamos por llave, se crea un iterable y reducimos sobre el iterable (contamos en este caso)

- data: 3
- engineering: 1
- big: 1
- systems: 1



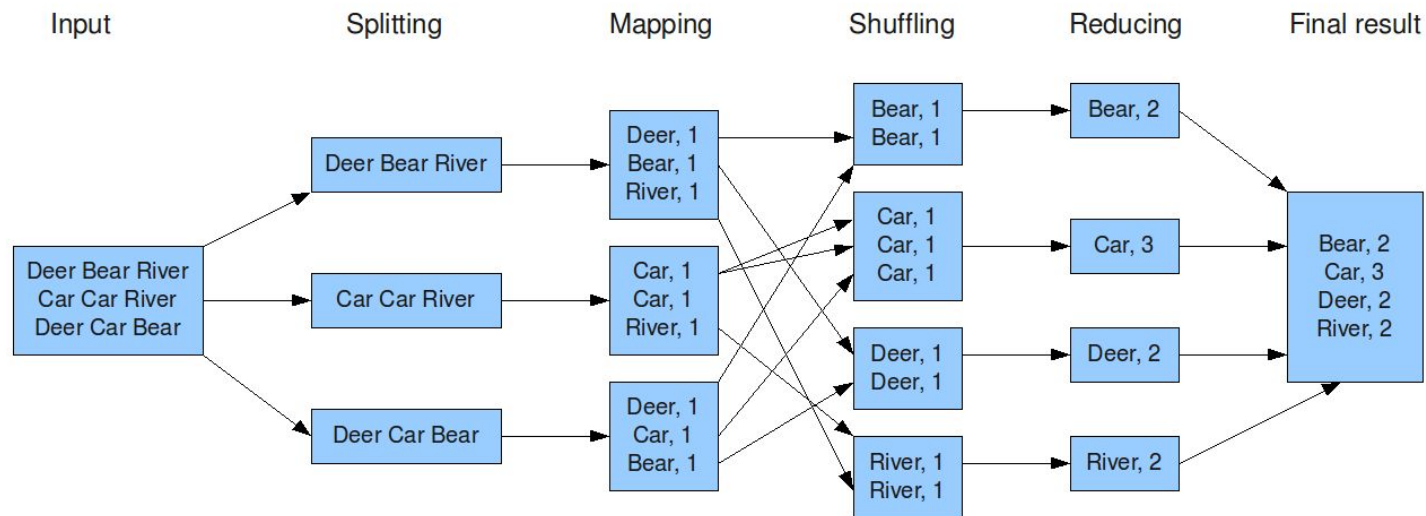
Ejemplo - Reduce

- Aplicamos la agregación
- En este caso, agrupamos por llave y sumamos valores
- Y volvemos a guardar la información
 - data: 3
 - engineering: 1
 - big: 1
 - systems: 1



Ejemplo

The overall MapReduce word count process



Map-Reduce

- Tenemos un nodo **coordinador** y nodos **workers**

Map-Reduce

- Tenemos un nodo **coordinador** y nodos **workers**
- Desbalance de los datos impacta en velocidad. ¿Qué pasa si hay una llave muy repetida? **Data skew**

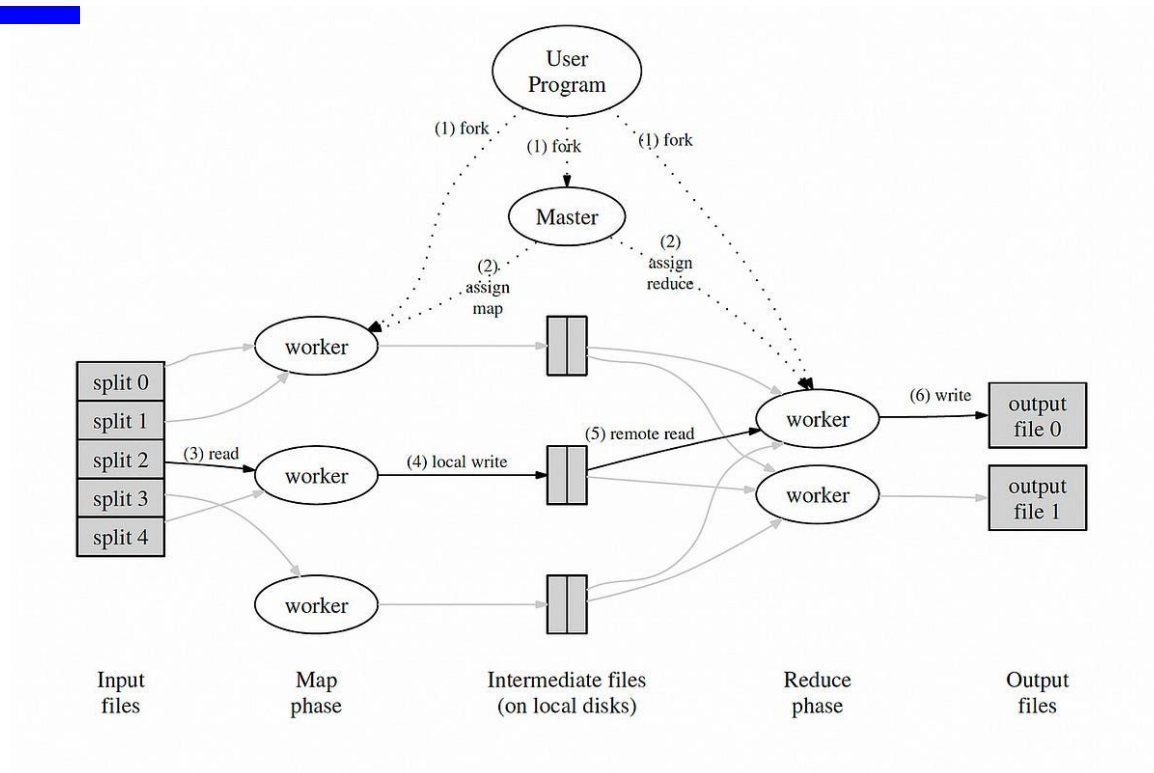
Map-Reduce

- Tenemos un nodo **coordinador** y nodos **workers**
- Desbalance de los datos impacta en velocidad. ¿Qué pasa si hay una llave muy repetida? **Data skew**
- Reducir datos antes del shuffle y asignación de buenas llaves ayuda a distribuir las cargas

Map-Reduce

- Tenemos un nodo **coordinador** y nodos **workers**
- Desbalance de los datos impacta en velocidad. ¿Qué pasa si hay una llave muy repetida? **Data skew**
- Reducir datos antes del shuffle y asignación de buenas llaves ayuda a distribuir las cargas
- Manejo de fallas en las distintas etapas

Map-Reduce



Map-Reduce

- Nosotros nos encargamos de definir las funciones, estructuras de datos y llaves
- El framework verá cómo maneja el sistema y realizar ejecuciones

Map-Reduce

- Nosotros nos encargamos de definir las funciones, estructuras de datos y llaves
- El framework verá cómo maneja el sistema y realizar ejecuciones
- Pero esto sigue siendo de bajo nivel para nuestro gusto...
- ¿Y si queremos iterar sobre resultados?

Ejemplos

- Proyecciones
- Filtros
- JOINS

```
# id, name, age, profession, birthday
users = [
    [1, "Alice", 30, "Engineer", "2000-01-15"],
    [2, "Bob", 24, "Artist", "2001-05-20"],
    [3, "Charlie", 35, "Engineer", "1995-11-01"],
    [4, "Diana", 29, "Artist", "2002-03-10"],
    [5, "Eve", 42, "Manager", "1998-07-22"]
]

# id, name, race, age, owner
pets = [
    [1, "Max", "Dog", 2, 1],
    [2, "Bella", "Cat", 1, 2],
    [3, "Charlie", "Dog", 3, 3],
    [4, "Lucy", "Cat", 2, 1],
    [5, "Cooper", "Bird", 1, 5]
]
```

(Py)Spark



Map-Reduce vs Spark

- Spark es un framework de más alto nivel que Map-Reduce
- Spark nos permite **trabajar en memoria** en vez de en disco
- Al trabajar en memoria, **iterar sobre resultados** es más barato
- Además de darnos mayor flexibilidad para algunas operaciones

PySpark

- API de Spark para Python
- Lazy evaluation
- **RDD**: Resilient Distributed Dataset

PySpark



`rdd.map`(función que aplicamos a cada elemento)

→ genera pares (llave, valor)

`rdd.reduceByKey`(función)

→ agrupa por llave

→ aplica función a los valores



PySpark



```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.getOrCreate()
```

```
sc = spark.sparkContext
```

```
text_file = sc.textFile("demo_file_spark.txt")
```

```
counts = text_file.flatMap(lambda line: line.split(" ")) \
    .map(lambda word: (word, 1)) \
    .reduceByKey(lambda a, b: a + b)
```

```
counts.collect()
```

Recapitulando

- Queremos un buscador con **resultados relevantes**: PageRank lo logra, pero necesitamos ejecutarlo eficientemente a gran escala
- MapReduce es un **paradigma y framework** para **paralelizar, distribuir y tolerar fallos en el procesamiento de datos**
- Apache Spark **extiende** MapReduce con ejecución en **memoria y APIs de más alto nivel**
- ◆ Próxima clase: profundizamos en PageRank e implementamos su lógica con MapReduce



PySpark

Nos vamos a las actividades (material en GitHub)

Lectura complementaria

- Paper Map-Reduce:
<https://static.googleusercontent.com/media/research.google.com/es//archive/mapreduce-osdi04.pdf>
- Libro Mining of Massive Datasets, capítulo 2: MapReduce and the new software stack
- Operadores PySpark:
<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.RDD.html>